

Hadoop与Spark

Spark安装、部署、与运行



Spark安装、部署、与运行

- ◆ Spark部署
 - ◆ 本安装采用Standalone部署模式，其它模式包括Spark on YARN、Spark on Mesos等不同的部署模式
 - ◆ 主要的区别是采用的资源管理器不一样

Spark安装、部署、与运行

- ◆ Spark部署
 - ◆ 事先要安装JDK1.8.0_144
 - ◆ 请参考“Hadoop安装与HDFS、MapReduce实验”一章

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 安装Python
- ◆ 从<https://www.continuum.io/downloads>下载Anaconda 2

```
cd /opt/linuxsir
```

```
ls
```

```
bash ./Anaconda2-4.4.0-Linux-x86_64.sh
```

\\可以指定安装路径为/opt/linuxsir/anaconda2

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 配置Python
- ◆ 编辑~/.bashrc文件，并且使之生效

```
echo "" >> ~/.bashrc
echo "export PYTHON_HOME=/opt/linuxsir/anaconda2" >> ~/.bashrc
echo "export PATH=\$PATH:\$PYTHON_HOME/bin" >> ~/.bashrc

cat ~/.bashrc
source ~/.bashrc
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 安装Scala
- ◆ 从<http://www.scala-lang.org/download/>下载scala-2.10.1

```
cd /opt/linuxsir  
tar xzvf ./scala-2.10.1.tgz  
  
mv scala-2.10.1/ /opt/linuxsir/scala
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 配置Scala
- ◆ 编辑~/.bashrc文件，并且使之生效

```
echo "" >> ~/.bashrc
echo "export SCALA_HOME=/opt/linuxsir/scala" >> ~/.bashrc
echo "export PATH=$PATH:$SCALA_HOME/bin:$SCALA_HOME/sbin" >>
~/.bashrc
```

```
cat ~/.bashrc
source ~/.bashrc
```

```
\\验证scala是否安装成功
scala -version
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 从<https://spark.apache.org/downloads.html>
- ◆ 下载spark-2.1.1
- ◆ 并且解压缩

```
cd /opt/linuxsir  
tar xzvf ./spark-2.1.1-bin-hadoop2.7.tgz  
mv spark-2.1.1-bin-hadoop2.7/ /opt/linuxsir/spark
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 编辑~/.bashrc文件，并且使之生效
- ◆ 备注：pyspark和py4j等两个模块，在Spark的安装目录里，需要定义PYTHONPATH环境变量

```
echo "" >> ~/.bashrc
echo "export SPARK_HOME=/opt/linuxsir/spark" >> ~/.bashrc
echo "export PATH=\$PATH:\$SPARK_HOME/bin:\$SPARK_HOME/sbin" >>
~/.bashrc
echo "export
PYTHONPATH=\$PYTHONPATH:\$SPARK_HOME/python/:\$SPARK_HOME/
python/lib/py4j-0.10.4-src.zip" >> ~/.bashrc

cat ~/.bashrc
source ~/.bashrc
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 在/opt/linuxsir/spark/conf目录下
- ◆ 从模板spark-env.sh.template拷贝spark-env.sh文件

```
cd /opt/linuxsir/spark/conf  
cp spark-env.sh.template spark-env.sh
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 编辑spark-env.sh文件，内容如下

```
export JAVA_HOME=/opt/linuxsir/java/jdk
export HADOOP_HOME=/opt/linuxsir/hadoop
export HADOOP_CONF_DIR=/opt/linuxsir/hadoop/etc/hadoop
export HIVE_HOME=/opt/linuxsir/hive
```

```
export SCALA_HOME=/opt/linuxsir/scala
export SCALA_LIBRARY_PATH=${SPARK_HOME}/lib
```

```
export SPARK_HOME=/opt/linuxsir/spark
export SPARK_DIST_CLASSPATH=$(hadoop classpath)
export SPARK_LIBRARY_PATH=${SPARK_HOME}/lib
```

```
export SPARK_MASTER_HOST=192.168.31.129
export SPARK_MASTER_PORT=17077 #7077被占用就改成17077
export SPARK_MASTER_WEBUI_PORT=18080 #8080被占用就改成18080
export SPARK_WORKER_WEBUI_PORT=18081 #8081被占用就改成18081
export SPARK_WORKER_MEMORY=1280m
export SPARK_LOCAL_DIRS=/opt/linuxsir/sparkdata
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 在/opt/linuxsir/spark/conf目录下
- ◆ 从模板拷贝并编辑slaves文件

```
cd /opt/linuxsir/spark/conf  
cp slaves.template slaves
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 用echo命令修改其内容，在/opt/linuxsir/spark/conf目录下的slaves文件

```
echo "" >> slaves  
echo "hd-slave1" >> slaves  
echo "hd-slave2" >> slaves  
cat slaves
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 从hd-master节点把~/.bashrc/python/scala/spark等配置文件和软件包(整个目录), 复制到hd-slave1和hd-slave2节点

```
scp -r ~/.bashrc root@192.168.31.130:~/.bashrc
```

```
scp -r ~/.bashrc root@192.168.31.131:~/.bashrc
```

```
scp -r /opt/linuxsir/anaconda2 root@192.168.31.130:/opt/linuxsir
```

```
scp -r /opt/linuxsir/anaconda2 root@192.168.31.131:/opt/linuxsir
```

```
scp -r /opt/linuxsir/scala root@192.168.31.130:/opt/linuxsir
```

```
scp -r /opt/linuxsir/scala root@192.168.31.131:/opt/linuxsir
```

```
scp -r /opt/linuxsir/spark root@192.168.31.130:/opt/linuxsir
```

```
scp -r /opt/linuxsir/spark root@192.168.31.131:/opt/linuxsir
```

Spark安装、部署、与运行

- ◆ Spark部署
- ◆ 在三个节点上运行`~/bashrc`，刷新环境变量

```
source ~/.bashrc  
ssh root@192.168.31.130 source ~/.bashrc  
ssh root@192.168.31.131 source ~/.bashrc
```

Spark安装、部署、与运行

- ◆ 启动Spark
- ◆ 首先需要启动hadoop

```
cd /opt/linuxsir/hadoop/sbin  
./start-dfs.sh  
./start-yarn.sh
```

```
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

```
\\停止Hadoop用如下命令  
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

Spark安装、部署、与运行

- ◆ 启动Spark
- ◆ 确认hive的配置文件/usr/local/hive/conf/hive-site.xml
- ◆ 已经有metastore的设置项

```
<property>  
  <name>hive.metastore.uris</name>  
  <value>thrift://192.168.31.129:19083</value>  
</property>
```

Spark安装、部署、与运行

- ◆ 启动Spark
- ◆ 用如下命令，启动hive metastore

```
cd /opt/linuxsir/hive  
./bin/hive --service metastore -p 19083 &
```

```
\\停止metastore用如下命令  
ps -ef|grep metastore  
kill -9 16336 \\16336是metastore进程号
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 在/opt/linuxsir/spark/conf目录下，配置spark的hive-site.xml。主要的目的是，
 - ◆ (1) 指出hive meta store所在的位置
 - ◆ (2) 配置spark thrift server host、和spark thrift server port(thrift server host /port也可以在启动thrift server时，在命令行设置)

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<configuration>
  <property>
    <name>hive.metastore.uris</name>
    <value>thrift://192.168.31.129:19083</value>
  </property>
</configuration>
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 启动spark

```
cd /opt/linuxsir/spark  
./sbin/start-all.sh
```

\\停止spark用如下命令

```
cd /opt/linuxsir/spark  
./sbin/stop-all.sh
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 启动spark
 - ◆ 问题1：如果启动Spark时，内存不足，解决办法如下
 - ◆ 编辑spark-env.sh文件，修改SPARK_WORKER_MEMORY配置项

```
export SPARK_WORKER_MEMORY=1280m
```

- ◆ 修改配置文件以后，要同步配置文件；然后再次启动Spark试试看

```
scp /opt/linuxsir/spark/conf/spark-env.sh root@192.168.31.130:/opt/linuxsir/spark/conf  
scp /opt/linuxsir/spark/conf/spark-env.sh root@192.168.31.131:/opt/linuxsir/spark/conf
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 启动spark
 - ◆ 查看进程是否正常启动，hd-master节点上应该有Master进程
 - ◆ hd-slave1和hd-slave2节点上应该有Worker进程
 - ◆ 当然，各个节点还应该启动了HDFS、YARN的相关进程

```
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 启动spark
 - ◆ 可以通过Web浏览器，对Spark作业进行监控

<http://192.168.31.129:18080/jobs>

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 启动spark
 - ◆ 进程及其关系

Spark 层↵	(1) beeline 连接到 thrift server↵ (2) thrift server 通过 metastore 查阅 MySQL 里的元信息↵ (3) thriftserver 把 SQL 查询交给 spark 运行 (Master/Slave)↵ (4) spark 可以存取本地文件，也可以存取 HDFS↵ ↵ Master↵	↵ ↵ ↵ ↵ ↵ ↵ ↵ Worker↵	↵ ↵ ↵ ↵ ↵ ↵ ↵ Worker↵	↵ ↵ ↵ ↵ ↵ ↵ ↵
Hive 层↵	Metastore↵ MySQL↵	↵	↵	↵
YARN 层↵	ResourceManager↵	NodeManager↵	NodeManager↵	...↵
HDFS 层↵	NameNode↵ Secondary NameNode↵	DataNode↵	DataNode↵	...↵
Hardware 各个节点↵	hd-master 节点↵ 192.168.31.129↵	hd-slave1 节点↵ 192.168.31.130↵	hd-slave2 节点↵ 192.168.31.131↵	...↵

Spark 和 Hadoop 的进程及其调用关系↵

pause



Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 1, Spark SQL Shell
 - ◆ 启动spark sql shell运行SQL

```
cd /opt/linuxsir/spark  
./bin/spark-sql --master spark://hd-master:17077
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 1, Spark SQL Shell
 - ◆ 在Spark SQL提示符下, 运行如下SQL

```
SET spark.sql.shuffle.partitions=20;
```

```
select id,word from words;
```

```
select id, count(*) from words group by id order by id;
```

```
select id,word from words order by id;
```

```
exit;
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 2, Spark pyspark Shell
 - ◆ 启动spark pyspark shell运行SQL

```
cd /opt/linuxsir/spark  
MASTER=spark://hd-master:17077 ./bin/pyspark
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 2, Spark pyspark Shell
 - ◆ 在PySpark提示符下运行如下代码。可以通过sqlContext.sql()运行其它SQL查询

```
from pyspark.sql import HiveContext
sqlContext=HiveContext(sc)
results=sqlContext.sql("show databases").collect()
print(results)
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 3, 用Pyspark shell进行数据处理
 - ◆ 从本地把/opt/linuxsir/test.txt文件拷贝到HDFS的/input目录。Spark默认从HDFS中读取数据

```
cd /usr/local/hadoop/bin  
./hdfs dfs -copyFromLocal /opt/linuxsir/test.txt /input
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 3, 用Pyspark shell进行数据处理
 - ◆ 启动pyspark shell

```
cd /opt/linuxsir/spark  
MASTER=spark://hd-master:17077 ./bin/pyspark
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 3, 用Pyspark shell进行数据处理
 - ◆ 在pyspark shell提示符下, 运行如下代码

```
from pyspark import SparkContext
sc = SparkContext()

textFile = sc.textFile("hdfs://hd-master:9000/input/test.txt")
textFile.count() # textFile.count()返回文件行数
textFile.first()
linesWithSpark = textFile.filter(lambda line: "hadoop" in line) # filter方法创建一个新的RDD数据集(包含有'Spark'的行)
linesWithSpark.count()

ctrl +D
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 4, scala shell 运行WordCount
 - ◆ 从HDFS读取文件并运行WordCount。
 - ◆ 从本地把/opt/linuxsir/test.txt文件拷贝到HDFS的/input目录

```
cd /opt/linuxsir/hadoop/bin  
./hdfs dfs -copyFromLocal /opt/linuxsir/test.txt /input
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 4, scala shell 运行WordCount
 - ◆ 运行scale shell

```
cd /opt/linuxsir/spark/bin  
MASTER=spark://hd-master:17077 ./spark-shell
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 4, scala shell 运行WordCount
 - ◆ 运行scale shell

```
val file = sc.textFile("hdfs://hd-master:9000/input/test.txt")  
val file = sc.textFile("file:///opt/linuxsir/test.txt")  
val count = file.flatMap(line => line.split(" ")).map(word => (word, 1)).reduceByKey(_+_)  
count.collect()  
  
:quit
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 5, scala shell运行SQL(本地文件)
 - ◆ 我们也可以在spark scala shell提示符下, 利用sqlContext.sql()运行sql

```
import org.apache.spark.sql.SQLContext
val sqlContext = new org.apache.spark.sql.hive.HiveContext(sc)

sqlContext.sql("CREATE TABLE IF NOT EXISTS words1 (id INT,word STRING) ROW FORMAT DELIMITED FIELDS
TERMINATED BY ' ' LINES TERMINATED BY '\n'")
sqlContext.sql("LOAD DATA LOCAL INPATH '/opt/linuxsir/hive-test.txt' OVERWRITE INTO TABLE words1")

sqlContext.sql("FROM words1 SELECT id, word").collect().foreach(println)
sqlContext.sql("select id, word FROM words1 order by id").collect().foreach(println)
val df = sqlContext.sql("SELECT * FROM words1")
df.show()

sqlContext.sql("insert into words1 values(7, 'tahao')")
val df = sqlContext.sql("SELECT * FROM words1")
df.show()
sqlContext.sql("drop table words1")

val df = spark.read.json("file:///opt/linuxsir/spark/examples/src/main/resources/people.json")
df.show()

:quit
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 6, scala shell运行SQL(HDFS文件)
 - ◆ 另一个实例如下, 在spark shell里面用利用sqlContext.sql()函数运行sql, 建表、插入数据、再进行查询
 - ◆ 表格的数据源为HDFS文件
 - ◆ 从本地把/opt/linuxsir/ hive-test.txt文件拷贝到HDFS的/input目录

```
cd /opt/linuxsir/hadoop/bin  
./hdfs dfs -put /opt/linuxsir/hive-test.txt /input
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 6, scala shell运行SQL(HDFS文件)
 - ◆ 启动scale shell

```
cd /opt/linuxsir/spark/bin  
MASTER=spark://hd-master:17077 ./spark-shell
```

Spark安装、部署、与运行

- ◆ 启动Spark
 - ◆ 6, scala shell运行SQL(HDFS文件)
 - ◆ spark scala shell 提示符下, 运行如下代码, 利用sqlContext.sql()运行sql

```
import org.apache.spark.sql.SQLContext
val sqlContext = new org.apache.spark.sql.hive.HiveContext(sc)
sqlContext.sql("CREATE TABLE IF NOT EXISTS words1 (id INT,word
STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ' ' LINES
TERMINATED BY '\n'")

sqlContext.sql("LOAD DATA INPATH '/input/hive-test.txt' OVERWRITE INTO
TABLE words1")

val df = sqlContext.sql("SELECT * from words1")

df.filter(df("id") > 3).show()

:quit
```

pause



Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 在配置ThriftServer的时候，至少要配置ThriftServer的主机名和端口，如果要使用Hive数据的话，还要提供Hive Metastore的uris。
 - ◆ 编辑/opt/linuxsir/spark/conf/hive-site.xml文件，保证有如下几个配置项

```
<configuration>
  <property>
    <name>hive.metastore.uris</name>
    <value>thrift://hd-master:19083</value>
  </property>
  <property>
    <name>hive.server2.thrift.port</name>
    <value>10013</value>
  </property>
  <property>
    <name>hive.server2.thrift.bind.host</name>
    <value>hd-master</value>
  </property>
</configuration>
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
- ◆ 如果要存取HDFS文件，需要首先启动Hadoop

```
cd /opt/linuxsir/hadoop/sbin  
./start-dfs.sh  
./start-yarn.sh
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
- ◆ 首先，启动hive metastore。

```
cd /opt/linuxsir/hive  
./bin/hive --service metastore -p 19083 &
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
- ◆ 接着启动spark

```
cd /opt/linuxsir/spark  
./sbin/start-all.sh
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 启动thriftserver。

```
\\ 用如下命令启动thriftserver
cd /opt/linuxsir/spark
./sbin/start-thriftserver.sh --master spark://hd-master:17077 \
--hiveconf hive.server2.thrift.bind.host hd-master \
--hiveconf hive.server2.thrift.port 10013

\\ 等待thriftserver启动
netstat -lanp | grep 10013 \\ 确认thriftserver已经在10013端口监听
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 在beeline里面，连接到thrift server

```
!connect jdbc:hive2://hd-master:10013  
\\用户名为root，密码为rootroot
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 在beeline里面, 运行sql

```
show databases;  
use default;
```

```
show tables;
```

```
select * from words;  
select id,count(*) from words group by id order by id;  
select id,count(*) from words1 group by id order by id;
```

```
!exit
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 可以在spark的作业界面看到，每个sql语句对应一个job，监控端口是缺省8080已经改为18080
 - ◆ 可以通过浏览器启动spark的作业监控界面

<http://192.168.31.129:18080/jobs>

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 按照逆序停止软件
 - ◆ 首先，停止thrift server

```
cd /opt/linuxsir/spark  
./sbin/stop-thriftserver.sh
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 按照逆序停止软件
 - ◆ 停止spark

```
cd /opt/linuxsir/spark  
./sbin/stop-all.sh
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 按照逆序停止软件
 - ◆ 停止hive metastore

```
netstat -ntlp|grep 19083  
kill -9 16336 \\16336是进程号
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 按照逆序停止软件
 - ◆ 停止hadoop hdfs/yarn

```
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

Spark安装、部署、与运行

- ◆ 配置和启动thrift server
 - ◆ 通过日志分析错误情况

- Spark启动或者运行中出现错误，可以分析其Log文件
- Spark上运行的每个job，它的log，在每个从节点的SPARK_HOME/work目录下，每个job的日志包括stdout和stderr两个文件，内容是job写到标准输出、和标准错误输出的内容

pause



Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
- ◆ 为了在Windows上用Eclipse调试Spark Java程序，需要安装必要的软件并进行一系列的配置
- ◆ 具体描述如下

(1) 在Windows上解压缩hadoop 2.7.3

一般把hadoop-2.7.3.tar.gz文件，解压缩到D:\hadoop-2.7.3目录下。

(2) 在Windows上解压缩spark-2.1.0

把spark-2.1.1-bin-hadoop2.7.tgz文件，解压缩到d:\spark-2.1.1-bin-hadoop2.7目录下。

(3) 安装hadoop winutils

把hadoop-2.7.3-winutils.zip文件，解压缩到D:\hadoop-2.7.3\bin目录下

该文件包含hadoop.dll & winutils.exe等文件(请参考4.18节了解如何下载该文件)

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
- ◆ 为了在Windows上用Eclipse调试Spark Java程序，需要安装必要的软件并进行一系列的配置。具体描述如下

(4) 设定环境变量

右键点击我的电脑图标，选择“属性→高级系统设置→环境变量→系统变量→新建”等命令和菜单(界面元素)序列，添加如下变量

HADOOP_HOME d:\hadoop-2.7.3

在“系统变量”对话框里，选择Path环境变量，选择“编辑”，增加一项为
d:\hadoop-2.7.3\bin

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
- ◆ 为了在Windows上用Eclipse调试Spark Java程序，需要安装必要的软件并进行一系列的配置。具体描述如下

(5) 在Windows上新建Eclipse Maven项目

启动Eclipse，通过“File→New→New Project”打开对话框，新建Maven Project项目类型。编辑项目的pom.xml文件，内容如下

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.pai</groupId>
  <artifactId>spark_demo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>spark_demo</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
```

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
- ◆ 项目的pom.xml文件，内容如下（继续）

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.apache.hadoop</groupId>
    <artifactId>hadoop-common</artifactId>
    <version>2.7.3</version>
  </dependency>

  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.10</artifactId>
    <version>2.1.1</version>
  </dependency>

  <dependency>
    <groupId>jdk.tools</groupId>
    <artifactId>jdk.tools</artifactId>
    <version>1.8</version>
    <scope>system</scope>
    <systemPath>C:/Program Files/Java/jdk1.8.0_131/lib/tools.jar</systemPath>
  </dependency>
</dependencies>

</project>
```

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (6) 配置Maven的源
 - ◆ 配置Maven使用国内源，这样在构建Maven项目的时候，速度更快
 - ◆ 具体是在D:\apache-maven-3.5.0\conf\setting.xml文件中，增加如下内容，作为第一个mirror(镜像)
 - ◆ 注意，Maven安装在D:\apache-maven-3.5.0目录下
 - ◆ 关于Maven的安装和配置方法，请读者参考附录

```
<mirror>
  <id>aliyun</id>
  <url>http://maven.aliyun.com/nexus/content/groups/public/</url>
  <mirrorOf>central</mirrorOf>
</mirror>
```

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (7) 编写主程序Java代码
 - ◆ 新建一个WordCount .java源文件，代码如下

```
package com.pai.spark_demo;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import org.apache.spark.api.java.function.VoidFunction;

import scala.Tuple2;
```

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (7) 编写主程序Java代码
 - ◆ 新建一个WordCount .java源文件，代码如下

```
public class WordCount {  
    public static void main(String[] args) {  
        SparkConf conf = new SparkConf().setMaster("local").setAppName("WordCountTest");  
        JavaSparkContext sc = new JavaSparkContext(conf);  
        String outputDir = args[0];  
  
        List<String> list = new ArrayList<String>();  
        list.add("1 1 2 a b");  
        list.add("a b 1 2 3");  
        JavaRDD<String> lines = sc.parallelize(list);
```

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (7) 编写主程序Java代码
 - ◆ 新建一个WordCount .java源文件，代码如下

```
// line split
JavaRDD<String> words = lines.flatMap(line -> Arrays.asList(line.split(" ")).iterator());

// pairs
JavaPairRDD<String, Integer> pairs = words.mapToPair(new PairFunction<String, String, Integer>() {
    public Tuple2<String, Integer> call(String s) throws Exception {
        return new Tuple2<String, Integer>(s, 1);
    }
});

// reduce
JavaPairRDD<String, Integer> counts = pairs.reduceByKey(new Function2<Integer, Integer, Integer>() {
    public Integer call(Integer x, Integer y) throws Exception {
        return x + y;
    }
});
```

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (7) 编写主程序Java代码
 - ◆ 新建一个WordCount .java源文件，代码如下

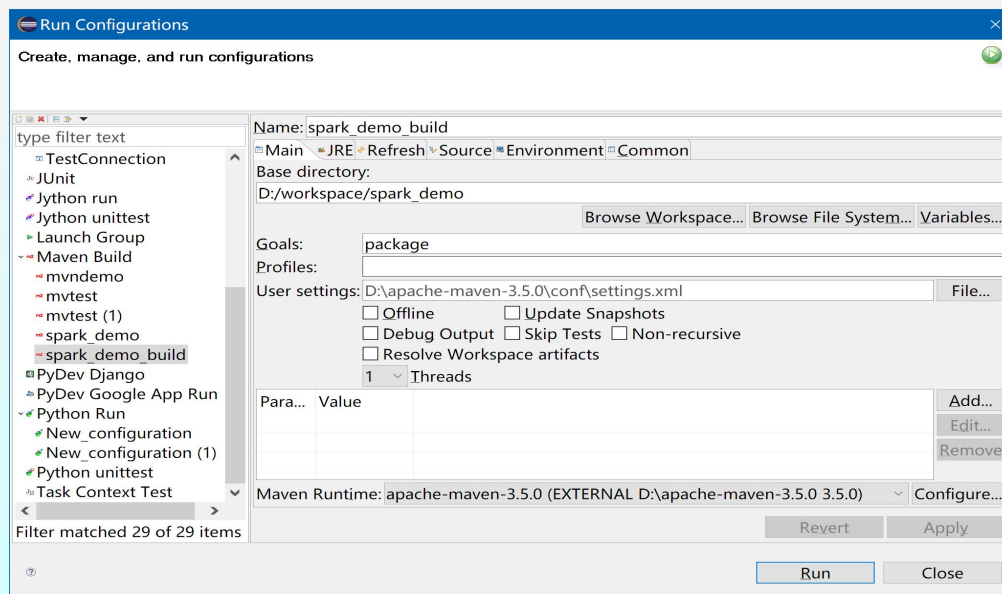
```
//output
System.out.println("The result is: "+counts.collect());
counts.saveAsTextFile(outputDir);

// output
//counts.foreach(new VoidFunction<Tuple2<String, Integer>>() {
//    public void call(Tuple2<String, Integer> tuple2) throws Exception {
//        System.out.println(tuple2);
//    }
//});

sc.stop();
}
```

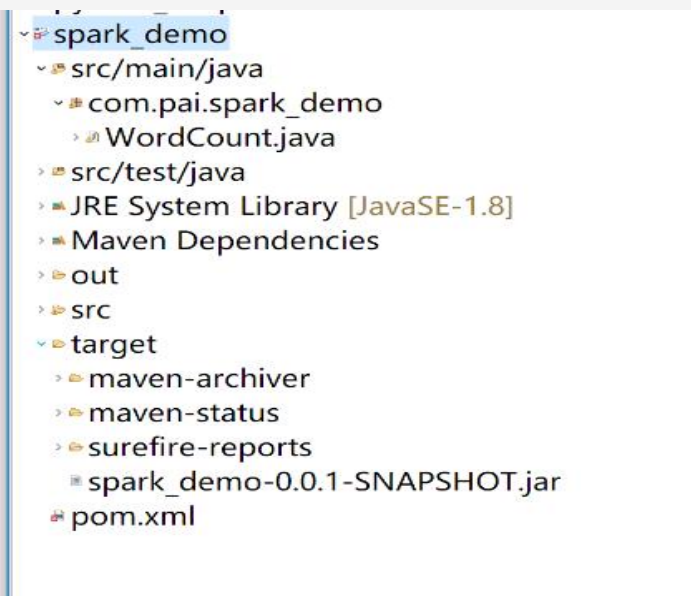
Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (8) 编译打包Maven项目
 - ◆ 选择Eclipse的项目，按右键，选择Run As，选择Run Configurations，打开对话框
 - ◆ 双击对话框左边的Maven Build，新建一个Run Configuration，命名为spark_demo_build，在Goals框里填写package，如图所示
 - ◆ 目的是编译和打包Maven项目



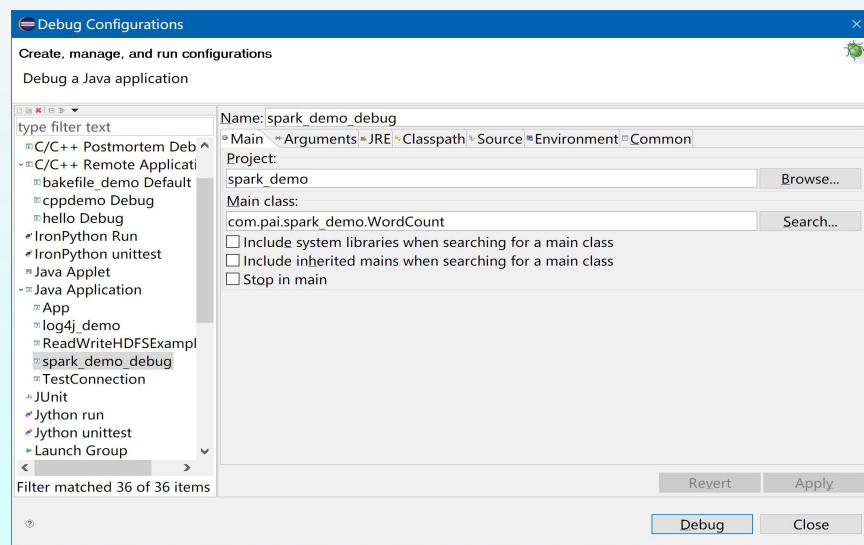
Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (8) 编译打包Maven项目
 - ◆ 最后选择Run按钮，编译和打包整个项目
 - ◆ 经过一会，在没有错误情况下，在项目的target目录下生成了编译后的Jar文件；如图所示



Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (9) 调试项目
 - ◆ 选择Eclipse的项目，按右键，选择Debug As，选择Debug Configurations，打开对话框
 - ◆ 双击对话框左边的Java Applications，新建一个Debug Configuration，命名为spark_demo_debug，设置Project为项目名称(应该已经设置好)，设置Main Class(可以点击Search按钮进行选择)
 - ◆ 设置Arguments为out，如图所示



Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (9) 调试项目
 - ◆ 选择Debug按钮，就可以对程序进行调试，如图所示

- 注意，由于在代码里新建SparkConf的时候，把Master属性设置为local，所以这里进行的是本地调试，不是真正的分布式运行和调试
- 如果需要在分布式的Spark集群中运行程序，并且对其进行调试，请参考相关资料

Spark安装、部署、与运行

- ◆ 在Windows上用Eclipse调试Spark Java程序
 - ◆ (10) 把结果写入文件
 - ◆ 如果在代码中，把结果写入out目录下的文件(请参考上述代码分析)，那么程序运行结束后，在项目根目录下，生成一个out文件夹，底下有两个文件，即_SUCCESS和part-00000。
 - ◆ 双击part-00000，打开该文件，可以看到其内容，如图所示

